

Seguro Viagem – Gestão de Apólices

Documentação técnica do código (versão .NET)

Carlos Guilherme Fontes Pereira

Teste técnico – CORIS

23 de setembro de 2026

Resumo

Este documento descreve o funcionamento de uma aplicação web para gestão de apólices de seguro viagem, construída com ASP.NET Core 8 (C#), Entity Framework Core, React 18 e MySQL 8, com implantação prevista na Microsoft Azure. São apresentados o modelo de dados, as regras de negócio, a organização do código em camadas, a API e os testes automatizados, com o objetivo de permitir que o leitor entenda o sistema e consiga modificá-lo.

Sumário

1	Introdução	2
1.1	Tecnologias	2
2	Modelo de dados	2
3	Regras de negócio	4
3.1	Cálculo do prêmio	4
3.2	Validações e demais regras	4
4	Arquitetura do backend	4
4.1	Exemplo: o que acontece ao emitir uma apólice	5
4.2	Program.cs e injeção de dependência	5
4.3	Validação	6
4.4	Controller	8
4.5	Service	10
4.6	Calculadora do prêmio	14
5	SOLID e Clean Code	15
6	API	16
7	Frontend	16

8	Testes	17
9	Execução e implantação	17
9.1	Com Docker	17
9.2	Na Azure	17
10	Trabalhos futuros	18

1 Introdução

A aplicação permite a um operador de seguradora **cadastrar, consultar, editar e excluir** apólices de seguro viagem. Ao preencher o formulário, o operador informa os dados do segurado, o destino, o plano e o período da viagem; o sistema então calcula o *prêmio*, isto é, o valor a ser pago pelo seguro.

O projeto está dividido em duas partes independentes que conversam por HTTP/JSON:

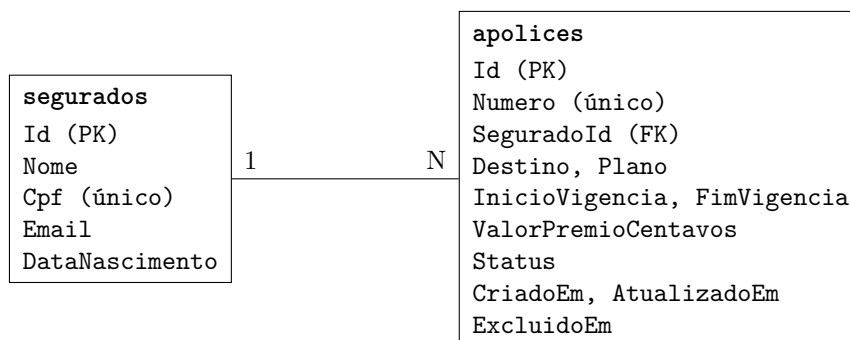
- **backend** (`backend/`): Web API em ASP.NET Core, responsável pelas regras e pelo banco;
- **frontend** (`frontend/`): interface em React, que apenas exibe dados e envia formulários.

1.1 Tecnologias

Camada	Tecnologia
Backend	C#, ASP.NET Core 8 Web API, Swagger
Acesso a dados	Entity Framework Core 8 (provider Pomelo para MySQL)
Frontend	React 18, Vite, React Router
Banco de dados	MySQL 8
Testes	xUnit, EF Core InMemory
Ambiente	Docker Compose
Nuvem	Microsoft Azure

2 Modelo de dados

O banco possui duas tabelas com relacionamento **1:N**: um segurado pode ter várias apólices, e cada apólice pertence a um único segurado.



As tabelas são geradas pelo Entity Framework a partir das classes **Segurado** e **Apolice** (abordagem *code first*). A migration foi criada com `dotnet ef migrations add CriarTabelas` e é aplicada automaticamente quando a API inicia (`db.Database.Migrate()` no `Program.cs`).

Três decisões merecem destaque:

1. **Valores em centavos.** O prêmio é guardado como `int` em centavos ($32370 = \text{R\$ } 323,70$). Tipos de ponto flutuante como `double` não representam decimais com exatidão ($0,1 + 0,2 = 0,30000000000000004$), o que é inaceitável para valores financeiros.

2. **Segurado separado da apólice.** O CPF é único, então uma mesma pessoa que contrata outro seguro é reaproveitada, sem duplicar dados.
3. **Exclusão lógica.** Excluir uma apólice apenas preenche `ExcluidoEm`. Um *filtro global* no `AppDbContext` (`HasQueryFilter`) esconde essas apólices de todas as consultas, mas o registro continua no banco para auditoria.

Código 1: `CorisSeguros.Api/Data/AppDbContext.cs`

```
1 using CorisSeguros.Api.Models;
2 using Microsoft.EntityFrameworkCore;
3
4 namespace CorisSeguros.Api.Data;
5
6 public class AppDbContext : DbContext
7 {
8     public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
9     {
10    }
11
12    public DbSet<Segurado> Segurados => Set<Segurado>();
13    public DbSet<Apolice> Apolices => Set<Apolice>();
14
15    protected override void OnModelCreating(ModelBuilder modelBuilder)
16    {
17        modelBuilder.Entity<Segurado>(entidade =>
18        {
19            entidade.ToTable("segurados");
20            entidade.Property(s => s.Nome).HasMaxLength(120).IsRequired();
21            entidade.Property(s => s.Cpf).HasMaxLength(11).IsFixedLength().
22            IsRequired();
23            entidade.Property(s => s.Email).HasMaxLength(150).IsRequired();
24            entidade.HasIndex(s => s.Cpf).IsUnique();
25        });
26
27        modelBuilder.Entity<Apolice>(entidade =>
28        {
29            entidade.ToTable("apolices");
30            entidade.Property(a => a.Numero).HasMaxLength(20).IsRequired();
31            entidade.Property(a => a.Destino).HasMaxLength(30).IsRequired();
32            entidade.Property(a => a.Plano).HasMaxLength(20).IsRequired();
33            entidade.Property(a => a.Status).HasMaxLength(20).IsRequired();
34            entidade.HasIndex(a => a.Numero).IsUnique();
35            entidade.HasIndex(a => a.Status);
36
37            // um segurado tem várias apólices (1:N)
38            entidade.HasOne(a => a.Segurado)
39                .WithMany(s => s.Apolices)
40                .HasForeignKey(a => a.SeguradoId);
41
42            // exclusão lógica: as consultas ignoram as apólices excluídas
43            entidade.HasQueryFilter(a => a.ExcluidoEm == null);
44        });
45    }
```

3 Regras de negócio

3.1 Cálculo do prêmio

$\text{prêmio} = \text{diária do plano} \times \text{dias} \times \text{fator do destino} \times \text{fator da idade}$

		Destino	Fator		
Plano	Diária			Idade	Fator
Essencial	R\$ 12,90	Nacional	50%	até 59	100%
Plus	R\$ 24,90	América do Sul	100%	60 a 74	160%
Premium	R\$ 39,90	Europa	130%	75 ou mais	250%
		América do Norte	140%		
		Ásia, África, Oceania	150%		

Os dias incluem o primeiro e o último (de 01/10 a 10/10 são 10 dias), e a idade é calculada na data de início da viagem. Os valores das tabelas ficam na classe `Models/Catalogo.cs`.

Exemplo. Plano Plus, Europa, 10 dias, segurado de 31 anos:

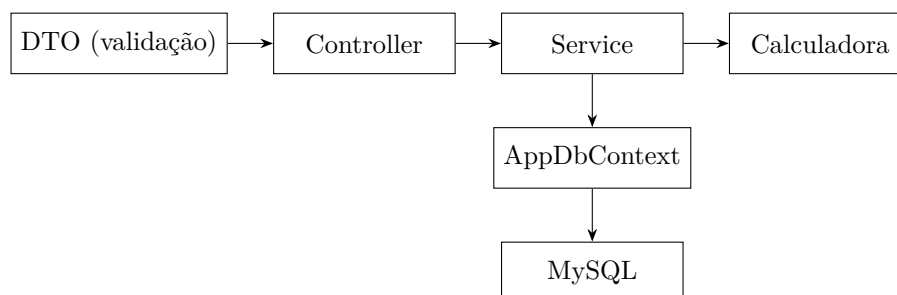
$$2490 \times 10 = 24900 \rightarrow 24900 \times 1,30 = 32370 \rightarrow 32370 \times 1,00 = 32370 \text{ centavos} = \text{R\$ } 323,70.$$

3.2 Validações e demais regras

- Todos os campos são obrigatórios e o CPF deve ter dígitos verificadores válidos.
- O fim da vigência não pode ser anterior ao início, e a viagem tem no máximo 365 dias.
- No cadastro, o início da vigência não pode ser anterior a hoje (na edição a apólice pode já estar em vigor).
- Toda apólice recebe um número único no formato `CRS-2026-XXXXXXX`.
- Ao cadastrar um CPF que já existe, o segurado é atualizado e reaproveitado.

4 Arquitetura do backend

O backend é organizado em camadas, e cada classe tem uma única responsabilidade:



Pasta / arquivo	Responsabilidade
Program.cs	configura banco, injeção de dependência, Swagger, CORS e formato dos erros
Controllers/	recebem a requisição HTTP e devolvem a resposta
Dtos/	formato de entrada (com as validações) e de saída da API
Services/	regras de negócio (<code>ApoliceService</code>) e cálculo do prêmio
Models/	entidades do banco e o catálogo de planos, destinos e status
Data/	<code>AppDbContext</code> , migrations e dados de exemplo
Validacoes/	atributo <code>[Cpf]</code>

4.1 Exemplo: o que acontece ao emitir uma apólice

1. O React envia `POST /api/apolices` com os dados do formulário em JSON.
2. O ASP.NET converte o JSON em um `ApoliceRequest` e executa as validações (atributos como `[Required]` e `[Cpf]`, e depois o método `Validate`). Havendo erro, a API responde 400 com uma mensagem por campo e o controller nem é chamado, por causa do atributo `[ApiController]`.
3. O `ApolicesController` verifica a data de início e chama `ApoliceService.Criar`.
4. O service cria ou atualiza o segurado pelo CPF, calcula o prêmio, gera o número da apólice e grava tudo com um único `SaveChangesAsync` (segurado e apólice são salvos juntos).
5. A apólice é convertida em `ApoliceResponse` e a resposta é 201 `Created`.

4.2 Program.cs e injeção de dependência

Código 2: `CorisSeguros.Api/Program.cs`

```

1 using System.Text.Json;
2 using CorisSeguros.Api.Data;
3 using CorisSeguros.Api.Services;
4 using Microsoft.AspNetCore.Mvc;
5 using Microsoft.EntityFrameworkCore;
6
7 var builder = WebApplication.CreateBuilder(args);
8
9 builder.Services.AddControllers()
10    .ConfigureApiBehaviorOptions(options =>
11    {
12        // Erros de validação no formato { message, errors: { campo: ["mensagem"]} }
13
14        options.InvalidModelStateResponseFactory = context =>
15        {
16            var erros = context.ModelState
17                .Where(campo => campo.Value!.Errors.Count > 0)
18                .ToDictionary(
19                    campo => JsonNamingPolicy.CamelCase.ConvertName(campo.Key),
20                    campo => campo.Value!.Errors.Select(e => e.ErrorMessage).
21                    ToArray());
22
23            return new BadRequestObjectResult(new { message = "Verifique os campos
informados.", errors = erros });
24        }
25    });

```

```

24
25 builder.Services.AddEndpointsApiExplorer();
26 builder.Services.AddSwaggerGen();
27
28 // Banco MySQL (a conexão fica no appsettings.json ou em variável de ambiente)
29 string conexao = builder.Configuration.GetConnectionString("Padrao");
30 builder.Services.AddDbContext<AppDbContext>(options =>
31     options.UseMySQL(conexao, new MySqlServerVersion(new Version(8, 0, 36)),
32     mysql => mysql.EnableRetryOnFailure())); // tenta de novo se o banco ainda
        estiver subindo
33
34 // Injeção de dependência: onde pedir ICalculadoraPremio, entrega a
        CalculadoraPremio
35 builder.Services.AddScoped<ICalculadoraPremio, CalculadoraPremio>();
36 builder.Services.AddScoped<ApoliceService>();
37
38 builder.Services.AddCors(options =>
39 {
40     options.AddDefaultPolicy(policy => policy.AllowAnyOrigin().AllowAnyHeader().
        AllowAnyMethod());
41 });
42
43 var app = builder.Build();
44
45 // Cria/atualiza as tabelas e cadastra os dados de exemplo
46 using (var scope = app.Services.CreateScope())
47 {
48     var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
49     db.Database.Migrate();
50
51     var service = scope.ServiceProvider.GetRequiredService<ApoliceService>();
52     await DadosIniciais.Popular(db, service);
53 }
54
55 app.UseSwagger();
56 app.UseSwaggerUI();
57
58 app.UseCors();
59
60 app.MapGet("/api/health", () => new { status = "ok" });
61 app.MapControllers();
62
63 app.Run();

```

A linha `AddScoped<ICalculadoraPremio, CalculadoraPremio>()` diz ao ASP.NET: quando uma classe pedir um `ICalculadoraPremio` no construtor, entregue um `CalculadoraPremio`. O `InvalidModelStateResponseFactory` padroniza os erros de validação no formato que o frontend espera: `{ message, errors: { campo: ["mensagem"] } }`.

4.3 Validação

As validações ficam no próprio DTO. Os atributos validam cada campo; o método `Validate` (da interface `IValidatableObject`) valida as regras que envolvem mais de um campo e só é executado se os atributos passarem.

Código 3: `CorisSeguros.Api/Dtos/ApoliceRequest.cs`

```

1 using System.ComponentModel.DataAnnotations;
2 using CorisSeguros.Api.Models;
3 using CorisSeguros.Api.Validacoes;
4
5 namespace CorisSeguros.Api.Dtos;
6
7 // Dados que o frontend envia para cadastrar, editar ou cotar uma apólice
8 public class ApoliceRequest : IValidatableObject
9 {
10     public const int VigenciaMaximaDias = 365;
11
12     [Required(ErrorMessage = "Campo obrigatório.")]
13     [StringLength(120, MinimumLength = 3, ErrorMessage = "Informe o nome completo
14     (3 a 120 caracteres).")]
15     public string? SeguradoNome { get; set; }
16
17     [Required(ErrorMessage = "Campo obrigatório.")]
18     [Cpf]
19     public string? SeguradoCpf { get; set; }
20
21     [Required(ErrorMessage = "Campo obrigatório.")]
22     [EmailAddress(ErrorMessage = "E-mail inválido.")]
23     [StringLength(150, ErrorMessage = "E-mail muito longo.")]
24     public string? SeguradoEmail { get; set; }
25
26     [Required(ErrorMessage = "Campo obrigatório.")]
27     public DateOnly? SeguradoNascimento { get; set; }
28
29     [Required(ErrorMessage = "Campo obrigatório.")]
30     public string? Destino { get; set; }
31
32     [Required(ErrorMessage = "Campo obrigatório.")]
33     public string? Plano { get; set; }
34
35     [Required(ErrorMessage = "Campo obrigatório.")]
36     public DateOnly? InicioVigencia { get; set; }
37
38     [Required(ErrorMessage = "Campo obrigatório.")]
39     public DateOnly? FimVigencia { get; set; }
40
41     // usado só na edição (ativa ou cancelada)
42     public string? Status { get; set; }
43
44     // validações que dependem de mais de um campo.
45     // O ASP.NET só chama este método se os atributos acima passarem.
46     public IEnumerable<ValidationResult> Validate(ValidationContext
47     validationContext)
48     {
49         DateOnly hoje = DateOnly.FromDateTime(DateTime.Today);
50
51         if (SeguradoNascimento > hoje)
52         {
53             yield return new ValidationResult("A data de nascimento não pode ser
54             futura.", new[] { nameof(SeguradoNascimento) });
55         }
56
57         if (Catalogo.BuscarDestino(Destino) == null)
58         {

```

```

56         yield return new ValidationResult("Destino inválido.", new[] { nameof(
Destino) });
57     }
58
59     if (Catalogo.BuscarPlano(Plano) == null)
60     {
61         yield return new ValidationResult("Plano inválido.", new[] { nameof(
Plano) });
62     }
63
64     if (Status != null && Catalogo.BuscarStatus(Status) == null)
65     {
66         yield return new ValidationResult("Status inválido.", new[] { nameof(
Status) });
67     }
68
69     if (FimVigencia < InicioVigencia)
70     {
71         yield return new ValidationResult("O fim da vigência deve ser igual ou
posterior ao início.", new[] { nameof(FimVigencia) });
72     }
73     else if (FimVigencia!.Value.DayNumber - InicioVigencia!.Value.DayNumber + 1
> VigenciaMaximaDias)
74     {
75         yield return new ValidationResult($"A vigência máxima é de {
VigenciaMaximaDias} dias.", new[] { nameof(FimVigencia) });
76     }
77 }
78 }

```

O CPF é validado pelo atributo `Validacoes/CpfAttribute.cs`: cada dígito verificador é obtido multiplicando os dígitos anteriores por pesos decrescentes, somando e calculando $(10 \cdot \text{soma}) \bmod 11$ (o resultado 10 vale 0).

4.4 Controller

O controller não contém regra de negócio: repassa os dados ao service e escolhe o código HTTP da resposta (200, 201, 204 ou 404).

Código 4: `CorisSeguros.Api/Controllers/ApolicesController.cs`

```

1 using CorisSeguros.Api.Dtos;
2 using CorisSeguros.Api.Services;
3 using Microsoft.AspNetCore.Mvc;
4
5 namespace CorisSeguros.Api.Controllers;
6
7 [ApiController]
8 [Route("api/apolices")]
9 public class ApolicesController : ControllerBase
10 {
11     private readonly ApoliceService _service;
12
13     public ApolicesController(ApoliceService service)
14     {
15         _service = service;
16     }

```

```

17
18 // GET api/apolices?busca=carlos&status=ativa&pagina=1
19 [HttpGet]
20 public async Task<ActionResult<ListaPaginada>> Listar(string? busca, string?
status, int pagina = 1)
21 {
22     return await _service.Listar(busca, status, pagina);
23 }
24
25 // GET api/apolices/resumo
26 [HttpGet("resumo")]
27 public async Task<ActionResult<ResumoResponse>> Resumo()
28 {
29     return await _service.Resumo();
30 }
31
32 // GET api/apolices/5
33 [HttpGet("{id:int}")]
34 public async Task<ActionResult<ApoliceResponse>> Buscar(int id)
35 {
36     ApoliceResponse? apolice = await _service.Buscar(id);
37     if (apolice == null)
38     {
39         return NaoEncontrada();
40     }
41
42     return apolice;
43 }
44
45 // POST api/apolices
46 [HttpPost]
47 public async Task<ActionResult<ApoliceResponse>> Criar(ApoliceRequest dados)
48 {
49     // só no cadastro: na edição a apólice pode já estar em vigor
50     if (dados.InicioVigencia < DateOnly.FromDateTime(DateTime.Today))
51     {
52         return BadRequest(new
53         {
54             message = "Verifique os campos informados.",
55             errors = new { inicioVigencia = new[] { "O início da vigência não
pode ser anterior a hoje." } },
56         });
57     }
58
59     ApoliceResponse apolice = await _service.Criar(dados);
60
61     return CreatedAtAction(nameof(Buscar), new { id = apolice.Id }, apolice);
62 }
63
64 // PUT api/apolices/5
65 [HttpPut("{id:int}")]
66 public async Task<ActionResult<ApoliceResponse>> Atualizar(int id,
ApoliceRequest dados)
67 {
68     ApoliceResponse? apolice = await _service.Atualizar(id, dados);
69     if (apolice == null)
70     {
71         return NaoEncontrada();

```

```

72     }
73
74     return apolice;
75 }
76
77 // DELETE api/apolices/5
78 [HttpDelete("{id:int}")]
79 public async Task<IActionResult> Excluir(int id)
80 {
81     bool excluiu = await _service.Excluir(id);
82     if (!excluiu)
83     {
84         return NaoEncontrada();
85     }
86
87     return NoContent();
88 }
89
90 // POST api/apolices/cotacao (calcula o prêmio sem salvar)
91 [HttpPost("cotacao")]
92 public ActionResult<CotacaoResponse> Cotar(ApoliceRequest dados)
93 {
94     return _service.Cotar(dados);
95 }
96
97 private NotFoundObjectResult NaoEncontrada()
98 {
99     return NotFound(new { message = "Apólice não encontrada." });
100 }
101 }

```

4.5 Service

É a classe central do sistema. O construtor recebe o `AppDbContext` e a *interface* `ICalculadoraPremio`, e não uma classe concreta.

Código 5: `CorisSeguros.Api/Services/ApoliceService.cs`

```

1 using CorisSeguros.Api.Data;
2 using CorisSeguros.Api.Dtos;
3 using CorisSeguros.Api.Models;
4 using CorisSeguros.Api.Validacoes;
5 using Microsoft.EntityFrameworkCore;
6
7 namespace CorisSeguros.Api.Services;
8
9 // Regras de negócio das apólices
10 public class ApoliceService
11 {
12     public const int PorPagina = 10;
13
14     private readonly AppDbContext _db;
15     private readonly ICalculadoraPremio _calculadora;
16
17     public ApoliceService(AppDbContext db, ICalculadoraPremio calculadora)
18     {
19         _db = db;

```

```

20     _calculadora = calculadora;
21 }
22
23 public async Task<ListaPaginada> Listar(string? busca, string? status, int
pagina)
24 {
25     IQueryable<Apolice> consulta = _db.Apolices.Include(a => a.Segurado);
26
27     if (!string.IsNullOrEmpty(busca))
28     {
29         string termo = busca.Trim();
30         string numeros = CpfAttribute.SomenteNumeros(termo);
31
32         consulta = consulta.Where(a =>
33             a.Numero.Contains(termo) ||
34             a.Segurado.Nome.Contains(termo) ||
35             a.Segurado.Email.Contains(termo) ||
36             (numeros != "" && a.Segurado.Cpf.Contains(numeros)));
37     }
38
39     if (!string.IsNullOrEmpty(status))
40     {
41         consulta = consulta.Where(a => a.Status == status);
42     }
43
44     int total = await consulta.CountAsync();
45     int totalPaginas = (int)Math.Ceiling(total / (double)PorPagina);
46     if (pagina < 1)
47     {
48         pagina = 1;
49     }
50
51     List<Apolice> apolices = await consulta
52         .OrderByDescending(a => a.CriadoEm)
53         .ThenByDescending(a => a.Id)
54         .Skip((pagina - 1) * PorPagina)
55         .Take(PorPagina)
56         .ToListAsync();
57
58     return new ListaPaginada
59     {
60         Data = apolices.Select(ApoliceResponse.De).ToList(),
61         Pagina = pagina,
62         TotalPaginas = totalPaginas,
63         Total = total,
64     };
65 }
66
67 public async Task<ResumoResponse> Resumo()
68 {
69     IQueryable<Apolice> ativas = _db.Apolices.Where(a => a.Status == "ativa");
70
71     return new ResumoResponse
72     {
73         Total = await _db.Apolices.CountAsync(),
74         Ativas = await ativas.CountAsync(),
75         PremioAtivasCentavos = await ativas.SumAsync(a => a.ValorPremioCentavos
),

```

```

76     };
77 }
78
79 public async Task<ApoliceResponse?> Buscar(int id)
80 {
81     Apolice? apolice = await _db.Apolices.Include(a => a.Segurado).
FirstOrDefaultAsync(a => a.Id == id);
82
83     return apolice == null ? null : ApoliceResponse.De(apolice);
84 }
85
86 public async Task<ApoliceResponse> Criar(ApoliceRequest dados)
87 {
88     Apolice apolice = new Apolice
89     {
90         Numero = GerarNumero(),
91         Segurado = await SalvarSegurado(dados),
92         Status = "ativa",
93         CriadoEm = DateTime.Now,
94     };
95     PreencherDados(apolice, dados);
96
97     _db.Apolices.Add(apolice);
98     await _db.SaveChangesAsync(); // salva o segurado e a apólice juntos
99
100     return ApoliceResponse.De(apolice);
101 }
102
103 public async Task<ApoliceResponse?> Atualizar(int id, ApoliceRequest dados)
104 {
105     Apolice? apolice = await _db.Apolices.Include(a => a.Segurado).
FirstOrDefaultAsync(a => a.Id == id);
106     if (apolice == null)
107     {
108         return null;
109     }
110
111     apolice.Segurado = await SalvarSegurado(dados);
112     PreencherDados(apolice, dados);
113     if (dados.Status != null)
114     {
115         apolice.Status = dados.Status;
116     }
117     apolice.AtualizadoEm = DateTime.Now;
118
119     await _db.SaveChangesAsync();
120
121     return ApoliceResponse.De(apolice);
122 }
123
124 public async Task<bool> Excluir(int id)
125 {
126     Apolice? apolice = await _db.Apolices.FirstOrDefaultAsync(a => a.Id == id);
127     if (apolice == null)
128     {
129         return false;
130     }
131

```

```

132         // não apaga do banco, só marca como excluída
133         apolice.ExcluidoEm = DateTime.Now;
134         await _db.SaveChangesAsync();
135
136         return true;
137     }
138
139     public CotacaoResponse Cotar(ApoliceRequest dados)
140     {
141         return new CotacaoResponse
142         {
143             ValorPremioCentavos = CalcularPremio(dados),
144             Dias = dados.FimVigencia!.Value.DayNumber - dados.InicioVigencia!.Value
145                 .DayNumber + 1,
146         };
147     }
148
149     private void PreencherDados(Apolice apolice, ApoliceRequest dados)
150     {
151         apolice.Destino = dados.Destino!;
152         apolice.Plano = dados.Plano!;
153         apolice.InicioVigencia = dados.InicioVigencia!.Value;
154         apolice.FimVigencia = dados.FimVigencia!.Value;
155         apolice.ValorPremioCentavos = CalcularPremio(dados);
156     }
157
158     // se o CPF já existe, atualiza e reaproveita o segurado
159     private async Task<Segurado> SalvarSegurado(ApoliceRequest dados)
160     {
161         string cpf = CpfAttribute.SomenteNumeros(dados.SeguradoCpf!);
162
163         Segurado? segurado = await _db.Segurados.FirstOrDefaultAsync(s => s.Cpf ==
164             cpf);
165         if (segurado == null)
166         {
167             segurado = new Segurado { Cpf = cpf };
168             _db.Segurados.Add(segurado);
169         }
170
171         segurado.Nome = dados.SeguradoNome!.Trim();
172         segurado.Email = dados.SeguradoEmail!.Trim().ToLower();
173         segurado.DataNascimento = dados.SeguradoNascimento!.Value;
174
175         return segurado;
176     }
177
178     private int CalcularPremio(ApoliceRequest dados)
179     {
180         return _calculadora.Calcular(
181             dados.Plano!,
182             dados.Destino!,
183             dados.InicioVigencia!.Value,
184             dados.FimVigencia!.Value,
185             dados.SeguradoNascimento!.Value);
186     }
187
188     // exemplo: CRS-2026-3B5CE1F5
189     private static string GerarNumero()

```

```

188     {
189         string codigo = Guid.NewGuid().ToString("N").Substring(0, 8).ToUpper();
190
191         return $"CRS-{DateTime.Now.Year}-{codigo}";
192     }
193 }

```

Pontos importantes:

- Include(a => a.Segurado) carrega o segurado junto com a apólice (*join*).
- A busca por CPF compara só os números, então funciona com ou sem máscara.
- A paginação usa Skip e Take, com 10 apólices por página.
- Excluir não remove a linha: só preenche ExcluidoEm.

4.6 Calculadora do prêmio

Código 6: CorisSeguros.Api/Services/ICalculadoraPremio.cs

```

1 namespace CorisSeguros.Api.Services;
2
3 public interface ICalculadoraPremio
4 {
5     // retorna o valor do prêmio em centavos
6     int Calcular(string plano, string destino, DateOnly inicio, DateOnly fim,
7     DateOnly dataNascimento);
8 }

```

Código 7: CorisSeguros.Api/Services/CalculadoraPremio.cs

```

1 using CorisSeguros.Api.Models;
2
3 namespace CorisSeguros.Api.Services;
4
5 // prêmio = diária do plano x dias x % do destino x % da idade
6 public class CalculadoraPremio : ICalculadoraPremio
7 {
8     public int Calcular(string plano, string destino, DateOnly inicio, DateOnly fim
9     , DateOnly dataNascimento)
10     {
11         Plano planoEscolhido = Catalogo.BuscarPlano(plano) ?? throw new
12         ArgumentException("Plano inválido.");
13         Destino destinoEscolhido = Catalogo.BuscarDestino(destino) ?? throw new
14         ArgumentException("Destino inválido.");
15
16         int dias = fim.DayNumber - inicio.DayNumber + 1;
17         int idade = CalcularIdade(dataNascimento, inicio);
18
19         int valor = planoEscolhido.ValorDiariaCentavos * dias;
20         valor = AplicarPercentual(valor, destinoEscolhido.PercentualRisco);
21         valor = AplicarPercentual(valor, PercentualIdade(idade));
22
23         return valor;
24     }
25
26     private static int PercentualIdade(int idade)

```

```

24     {
25         if (idade >= 75)
26         {
27             return 250;
28         }
29
30         if (idade >= 60)
31         {
32             return 160;
33         }
34
35         return 100;
36     }
37
38     // idade na data de início da viagem
39     private static int CalcularIdade(DateOnly nascimento, DateOnly data)
40     {
41         int idade = data.Year - nascimento.Year;
42         if (nascimento > data.AddYears(-idade))
43         {
44             idade--;
45         }
46
47         return idade;
48     }
49
50     // soma 50 antes de dividir para arredondar para o centavo mais próximo
51     private static int AplicarPercentual(int centavos, int percentual)
52     {
53         return (centavos * percentual + 50) / 100;
54     }
55 }

```

O arredondamento usa apenas inteiros: $(\text{centavos} * \text{percentual} + 50) / 100$. Em C#, a divisão entre dois `int` descarta a parte decimal; somar 50 antes arredonda para o centavo mais próximo.

5 SOLID e Clean Code

Princípio	Aplicação
S – Responsabilidade única	validação (DTO), HTTP (Controller), regra de negócio (Service) e cálculo (Calculadora) em classes distintas
O – Aberto/fechado	nova regra de preço é uma nova classe que implementa <code>ICalculadoraPremio</code> ; novo plano é um novo item no <code>Catalogo</code>
L – Substituição de Liskov	qualquer implementação da calculadora pode substituir a atual
I – Segregação de interfaces	a interface da calculadora possui um único método
D – Inversão de dependência	o service depende da interface; o <code>Program.cs</code> escolhe a implementação

Quanto a Clean Code: os nomes seguem o vocabulário do negócio (apólice, segurado, prêmio, vigência), os métodos são curtos e os valores fixos ficam em constantes (`VigenciaMaximaDias`, `PorPagina`) ou no `Catalogo`.

6 API

A API pode ser testada pelo Swagger em <http://localhost:8000/swagger>.

Método	Rota	Descrição
GET	/api/apolices	lista paginada (busca, status, pagina)
GET	/api/apolices/resumo	totais da tela inicial
GET	/api/apolices/{id}	detalhe
POST	/api/apolices	cadastro (201)
PUT	/api/apolices/{id}	edição, com recálculo do prêmio
DELETE	/api/apolices/{id}	exclusão lógica (204)
POST	/api/apolices/cotacao	calcula o prêmio sem salvar
GET	/api/opcoes	planos, destinos e status

Exemplo de requisição e resposta de erro:

```
POST /api/apolices
{ "seguradoNome": "Carlos Pereira", "seguradoCpf": "123.456.789-00",
  "seguradoEmail": "carlos@email.com", "seguradoNascimento": "1995-05-10",
  "destino": "europa", "plano": "plus",
  "inicioVigencia": "2026-10-01", "fimVigencia": "2026-10-10" }

400 Bad Request
{ "message": "Verifique os campos informados.",
  "errors": { "seguradoCpf": ["CPF inválido."] } }
```

O C# usa nomes em PascalCase (SeguradoNome) e o JSON sai em camelCase (seguradoNome), que é o padrão do ASP.NET.

7 Frontend

Pasta	Conteúdo
src/pages	telas: lista, detalhe, cadastro e edição
src/components	formulário, paginação, diálogo de confirmação, notificações
src/api	client.js (função request) e apolices.js (uma função por endpoint)
src/hooks	useApolice e useOpcoes (busca de dados)
src/utils/format.js	formatação de moeda, datas e máscara de CPF

Toda chamada à API passa por `request`, que define os cabeçalhos, converte o corpo para JSON e, em caso de erro, lança um `ApiError` com a mensagem de cada campo, exibida abaixo do campo correspondente no formulário. Campos vazios são enviados como `null`, para que a API responda “Campo obrigatório.”.

A **cotação em tempo real** funciona assim: quando todos os campos estão preenchidos, o formulário aguarda 400ms sem alterações (*debounce*) e chama `/api/apolices/cotacao`. Se o usuário continuar digitando, a requisição anterior é cancelada com `AbortController`.

8 Testes

São 26 testes automatizados (xUnit), executados com `dotnet test` na pasta `backend`.

- `CalculadoraPremioTests`: combinações de plano, destino e idade (`[Theory]` com `[InlineData]`).
- `CpfTests`: CPFs válidos e inválidos.
- `ApoliceRequestTests`: campos obrigatórios, CPF e plano inválidos e limites da vigência.
- `ApoliceServiceTests`: cadastro, reaproveitamento do segurado, edição com recálculo, exclusão lógica, busca e paginação, resumo e cotação.

Os testes do service usam o banco *InMemory* do Entity Framework, então não precisam de MySQL:

```
var options = new DbContextOptionsBuilder<AppDbContext>()
    .UseInMemoryDatabase(Guid.NewGuid().ToString())
    .Options;
```

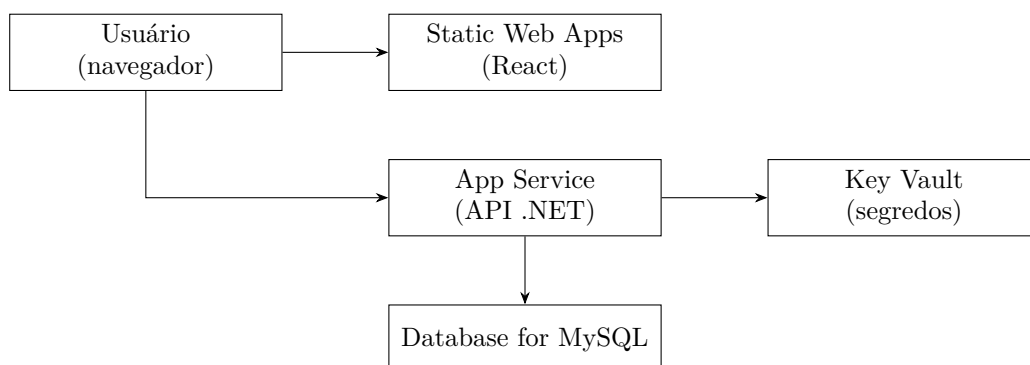
9 Execução e implantação

9.1 Com Docker

```
docker compose up -d --build
```

Aplicação em `http://localhost:3000` e Swagger em `http://localhost:8000/swagger`. Ao iniciar, a API aplica a migration e, se o banco estiver vazio, cadastra 5 apólices de exemplo.

9.2 Na Azure



- **Static Web Apps** hospeda o React compilado.
- **App Service** executa a API .NET 8.
- **Azure Database for MySQL** armazena os dados.
- **Key Vault** guarda a connection string do banco, lida pelo App Service via *Managed Identity*.

10 Trabalhos futuros

- Autenticação e perfis de acesso (JWT).
- Endosso: histórico das alterações de cada apólice.
- Testes de integração da API e testes no frontend.